

## CoreOps Week 5 Assignments

**Objective:** Understand how Kubernetes workloads behave during failures, how to design graceful shutdown, how backups & restore workflows operate, and how to prepare clusters for multi-zone resilience.

---

### MONDAY: PDBs, preStop Hooks & Graceful Shutdown

**Objective:** Understand how Kubernetes gracefully terminates pods and how to prevent downtime during node drains or rollouts.

---

#### SETUP

Deploy a simple web app: `kubectl create ns week5`

`kubectl run web --image=nginx -n week5`

Expose it: `kubectl expose pod web --port=80 -n week5`

#### LAB 1: Create a PodDisruptionBudget

##### Step 1: Convert the pod → a deployment

`kubectl delete pod web -n week5`

`kubectl create deployment web --image=nginx -n week5`

`kubectl scale deploy web --replicas=3 -n week5`

## Step 2: Create PDB (minAvailable 2)

```
cat <<EOF | kubectl apply -n week5 -f -
apiVersion: policy/v1
kind: PodDisruptionBudget
Metadata:
  name: web-pdb
Spec:
  minAvailable: 2
  Selector:
    matchLabels:
      app: web
EOF
```

## Step 3: Test a disruption

```
kubectl drain <any-node> --ignore-daemonsets --delete-emptydir-data
```

### Observe

- Only 1 pod is evicted
- The remaining 2 pods stay running
- Drain stops (PDB blocks eviction)

**Why:** This teaches how Kubernetes protects availability.

## LAB 2: Add a preStop Hook (graceful shutdown)

### Step 1: Patch deployment

```
kubectl patch deploy web -n week5 --type='json' -p=[
{"op": "add", "path": "/spec/template/spec/containers/0/lifecycle", "value": {
  "preStop": {"exec": {"command": ["/bin/sh", "-c", "echo Shutting down; sleep 10"]}} }]'
```

### Step 2: Delete one pod

```
kubectl delete pod -l app=web -n week5
```

### Observe

```
kubectl describe pod <pod>
```

Look at:

- Termination grace period
- Logs showing "Shutting down"
- Pod stays in "Terminating" for ~10s

**Why:** Real apps need time to drain connections.

### **LAB 3: Watch Graceful Shutdown Timeline**

`kubectll logs -f <old-pod>`

Watch events:

- SIGTERM sent
- preStop hook executed
- Pod removed from endpoints BEFORE killed

### **Why**

You learn real termination behavior.



## TUESDAY: Velero, VolumeSnapshots & Restore

**Objective:** Learn to back up and restore Kubernetes objects & volumes.

### SETUP

Install Minio (local S3 replacement): `kubectl apply -f https://raw.githubusercontent.com/vmware-tanzu/velero/master/examples/minio/00-minio-deployment.yaml -n velero`

Install Velero:

```
velero install \
  --provider aws \
  --bucket velero \
  --secret-file ./credentials-velero \
  --use-restic \
  --backup-location-config
region=minio,s3ForcePathStyle=true,s3Url=http://<minio-service-ip>:9000
```

### LAB 1: Create a sample app with PVC

`kubectl apply -n week5 -f https://k8s.io/examples/application/wordpress/wordpress-deployment.yaml`

This deploys PVC + MySQL + WordPress.

### LAB 2: Take a Velero backup

```
velero backup create wp-backup --include-namespaces week5
Check status: velero backup describe wp-backup --details
```

### LAB 3: Delete namespace

```
kubectl delete ns week5
```

### LAB 4: Restore

```
velero restore create wp-restore --from-backup wp-backup
```

Observe: `kubectl get pods -n week5`

**Why:** You experience full cluster recovery workflow.

### LAB 5: Snapshot a PVC

Create snapshot class (if cloud provider): `kubectl get volumesnapshotclass`

Create snapshot:

```
kubectl apply -f - <<EOF
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
Metadata:
name: wp-snap
namespace: week5
Spec:
Source:
persistentVolumeClaimName: wp-pvc
volumeSnapshotClassName: csi-hostpath-snapclass
EOF
```

Restore snapshot: kubectl apply -f snapshot-restore.yaml

---



## WEDNESDAY: Chaos Basics: Pod Kill & Latency Injection

**Objective:** Learn safe chaos experiments to observe system behavior.

### SETUP

Recreate your nginx deployment: `kubectl create deploy chaos-web --image=nginx -n week5 || kubectl scale deploy chaos-web --replicas=3 -n week5`

### LAB 1: Pod Kill Experiment (no tools)

#### Step 1: Delete random pods

`kubectl delete pod -l app=chaos-web -n week5 --force --grace-period=0`

#### Step 2: Observe rescheduling

`kubectl get pods -n week5 -w`

#### Why

Understand how Kubernetes self-heals.

### LAB 2: Add CPU latency (simple)

`kubectl exec -it <pod-name> -n week5 -- sh -c "yes > /dev/null &"` Observe CPU spike: `kubectl top pod -n week5`

**Why:** Simulates CPU throttling incidents.

### LAB 3: Network latency

Install tc inside pod: `kubectl exec -it <pod> -- apt update && apt install iproute2 -y`  
`kubectl exec -it <pod> -- tc qdisc add dev eth0 root netem delay 500ms`

Test: `curl <pod-ip>`

Remove latency: `kubectl exec -it <pod> -- tc qdisc del dev eth0 root netem`

## THURSDAY: Multi-Zone Architecture & Workload Spreading

**Objective:** Learn how to spread workloads across zones safely.

### SETUP

Check your node zones: `kubectl get nodes --show-labels | grep`  
[topology.kubernetes.io/zone](https://topology.kubernetes.io/zone)

### LAB 1: Create a deployment without spread constraints

```
kubectl create deploy zone-test --image=nginx -n week5
kubectl scale deploy zone-test --replicas=6 -n week5
Check placement: kubectl get pods -o wide -n week5
Most pods will go to the same zone.
```

### LAB 2: Apply Topology Spread Constraints

```
Patch deployment: kubectl apply -n week5 -f - <<EOF
apiVersion: apps/v1
kind: Deployment
Metadata:
  name: zone-test
Spec:
  replicas: 6
Selector:
  matchLabels: app: zone-test
Template:
  Metadata:
  Labels:
    app: zone-test
  Spec:
    topologySpreadConstraints:
      - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: ScheduleAnyway
    labelSelector:
      matchLabels:
        app: zone-test
    containers:- name: nginx
      image: nginx
EOF
```

Observe again: `kubectl get pods -o wide -n week5`  
Pods will now be evenly distributed.

## FRIDAY: Runbooks, Escalation Chains & Failover Drills

**Objective:** Learn operational discipline: how to define, document and execute failover.

### LAB 1: Create a Runbook Template

Create a file: nano [runbook-web-failure.md](#)

Add sections:

- User Impact Summary
- Symptoms
- Immediate Checks
- Diagnostics
- Root Cause Isolation
- Remediation Steps
- Recovery Verification
- Post-Incident Notes

### LAB 2: Create Your Escalation Chain

Example:

- L1 → Junior DevOps
- L2 → Senior DevOps
- L3 → Platform SRE
- L4 → Cloud Architect

Document:

- When to escalate
- What data must be shared
- Who owns what

### LAB 3: Failover Drill

#### Step 1: Deploy two versions:

```
kubectl create deploy app-v1 --image=nginx -n week5  
kubectl create deploy app-v2 --image=nginx -n week5
```

**Step 2: Route traffic to v1**

```
kubectrl expose deploy app-v1 --port=80 -n week5
```

**Step 3: Simulate failure of v1**

```
kubectrl scale deploy app-v1 --replicas=0 -n week5
```

**Step 4: Failover to v2**

```
kubectrl expose deploy app-v2 --port=80 --name=app-service -n week5
```

